Validacija rešenja



Organizacija rada

Problem	Teorijske osnove	Rešenje	Evaluacija rešenja	Rezultati/Diskusija

Cilj evaluacije

- Jasno je izraženo šta je to što želimo da pokažemo evaluacijom (šta je poželjan/nepoželjan ishod evaluacije)
  - Aplikativna rešenja – želimo da pokažemo da su ispunjeni ciljevi krajnjih korisnika
  - Tehnička rešenja – želimo da pokažemo da zaista rešavamo identifikovane tehničke nedostatke postojećih rešenja
  - Mašinsko učenje – želimo da pokažemo da je naše rešenje dovoljno dobro da se može koristiti u praksi

Protokol evaluacije

- Moguće bi bilo reprodukovati eksperiment na osnovu njegovog opisa
  1) Definisana je procedura eksperimenta
  2) Opisane su metrike performansi

Aplikativna rešenja

- Rad sa ljudima (*usability study*)

- Potrebno je proveriti da li aplikacija podržava ciljeve krajnjih korisnika (identifikovane u poglavlju „Teorijske osnove“)

- Definišite protokol testiranja aplikacije (kako izgleda sesija sa korisnikom)
  - Svaki *Use-case* je cilj koji korisnik želi da ispuni
  - Korisnik dobija aplikaciju i zadatak da postigne određeni cilj
  - Dok korisnik interaguje sa aplikacijom, posmatrate korisnika i identifikujete situacije u kojima se korisnik zbuni. Par sekundi razmišljanja nije problem, ali je problem ako korisnik ne zna šta dalje da uradi
  - Na kraju postizanja svakog cilja, korisnik dobija par pitanja. Na primer:
    - Da na Likertovoj skali (1-5) oceni koliko mu je teško bilo da postigne cilj
    - Nestrukturirana (*open-ended*) pitanja gde korisnik može da daje predloge za poboljšanje

- Testirajte sa 2-3 ljudi (ako je moguće, angažujte ljude koji nisu programeri)

Tehnička rešenja

- U poglavlju „Teorijske osnove“ ste identifikovali nedostatke postojećih rešenja koje želite da rešite
  - Na primer, predlažete NoSQL kao rešenje za određene nedostatke SQL
- Na osnovu toga ste definisali zahteve koje rešenje treba da ispuni
  - Na primer, performansa operacija čitanja/pisanja
- Sprovodite validaciju da li vaše rešenje zaista ispunjava zahteve (rešava nedostatke koje ste identifikovali)
- Na primer, definišete protokol testiranja performansi
  - Na primer, generišete 10^6 entiteta i poredite vreme odziva (1) vaše aplikacije i (2) određene SQL baze

TABELA II ZAHTEVI ALATA ZA UČENJE VEŠTINE PISANJA ODRŽIVOG KODA

Cilj interesne grupe	Zahtev	Argument kako zahtev doprinosi cilju
Studenti softverskog inženjerstva žele efektivno da savladaju veštinu pisanja održivog koda kako bi unapredili svoje karijere	1. Alat treba da simulira situacije iz realnog sveta koje zahtevaju upotrebu veštine pisanja održivog koda	Realistično okruženje povećava angažovanost u učenju i rezultate učenja, pomažući učenicima da razviju veštine potrebne za dato realistično okruženje [1]. Na primer, pitanje sa višestrukim izborom je manje realistično od programskog zadatka koji se rešava u integrisanom razvojnom okruženju
	2. Alat treba da automatski identifikuje nedostatke u znanju svakog studenta	Rešenje treba da automatski identifikuje nedostatke u znanju svakog studenta kako bi se omogućilo automatsko prilagođavanje instrukcije
	3. Alat treba da se automatski prilagodi nedostacima u znanju studenta odabirom adekvatnih zadataka i povratnih informacija	Rešenje treba da se adaptira nedostacima u znanju svakog studenta kako bi se optimizovali dobici u učenju i time efikasno razvile veštine učenika.
Instruktori teže donošenju odluka zasnovanih na podacima kako bi poboljšali iskustvo učenja za sebe i studente	4. Alat treba da nadgleda i prikazuje progres svakog studenta	Pomoću pregleda aktivnosti svakog učenika, instruktor može da usmeri svoju pažnju na studente kojima treba pomoć, dok alat obučava ostale studente.
	5. Alat treba da prati interakcije studenata sa edukativnim sadržajem da bi podržao analizu kvaliteta sadržaja	Pomoću ove funkcionalnosti, instruktori mogu da analiziraju studentske odgovore na zadatke, vreme provedeno u svakom segmentu ili šablone u učenju. Na taj način instruktori razumeju kojim delovima procesa učenja i edukativnih materijala je potrebno unapređenje. Specifično za održivost koda, instruktor može da analizira predat kod da bi utvrdio kršenje principa, preporuka i heuristika i adekvatno razvio model veština pisanja održivog koda.

Radovi orijentisani na mašinsko učenje

- Protokol testiranja. Na primer:
  - Podela skupa podataka na trening, validacioni i test skup u odnosu 80/10/10
  - Naglasite da li je podela stratifikovana
  - Definišite metrike performansi i njihove dopustive vrednosti

- Primeri metrike performansi:
  - Klasifikacija – *F*-mera
  - Klasterovanje – kvantitativne metrike poput *Silhouette Coefficient* i kvalitativna analiza dobijenih klastera
  - Interpretabilnost

- Analiza grešaka modela